

Classification des exercices



Exercices qui développe des automatismes

Ces exercices sont supers simples, et volontairement répétitifs. Ces exercices servent à répéter des syntaxes simples de façon à ce qu'elles deviennent des automatismes. Si vous arrivez à faire ces exercices rapidement, sans regarder vos notes, vous maîtrisez ce type d'exercices, sinon, continuez à répéter.



Exercices d'exploration

Ces exercices ne sont pas difficiles en soi, mais vous amène à tester votre compréhension des notes de cours. Après une révision des notes de cours, ces exercices devraient se faire facilement.



Exercices de dépassement

Ces exercices sont difficiles et ont pour but de vous amener comprendre les notions enseignées en profondeur et de développer les concepts plus avancés associés avec les structures montrés en classe et en exemple. Même avec les notes de cours ces exercices peuvent être difficiles. Si vous restez bloqués, donner un coup d'oeil à la solution, mais assurez-vous de les maîtriser avant l'examen.

Série d'exercices de base



Ex1 – base_1

Écrivez un programme qui affiche “Bonjour tout le monde” à l’écran.



Ex2 – base_2

Écrivez un programme qui affiche à l’écran:

Horaire:

7h00 - déjeuner

12h00 - diner

17h00 - souper



Ex3 – base_3

Écrivez un programme qui demande à l’usager d’entrer un entier, le programme affiche ensuite deux fois (2x) ce nombre.

Ex: Nombre entré: 10

Nombre affiché: 20

Série d'exercices de base



Ex4 – base_4

Écrivez un programme qui demande à l'utilisateur d'entrer un nombre, le programme affiche ensuite le nombre au carré.



Ex5 – base_5

Écrivez un programme qui calcule le prix à payer pour une série d'article en plus des taxes. Il demande à l'utilisateur le nombre d'articles achetés (tous les articles sont pareil, par exemple 5 bananes), le prix d'un article (une banane), puis il affiche le prix total en comptant 15% de taxe de vente.



Ex6 – base_6

Écrivez un programme qui affiche le nombre pi avec différente résolution. Pour obtenir la valeur de pi, importez le package math de façon statique et utiliser PI, il s'agit d'une constante défini dans le système (<http://tutorials.jenkov.com/java/math-operators-and-math-class.html#math-pi>):

- a) 3.14
- b) 3.141593
- c) 3.1415926536

Série d'exercices sur le IF



Ex1 – if_1

Écrivez un programme qui demande à l'utilisateur d'entrer un nombre. Le programme doit afficher si le nombre est positif, négatif ou nul (égal à zéro)



Ex2 – if_2

Écrivez un programme qui demande à un utilisateur d'entrer son poids (en kg) et sa taille (en m). Le programme doit ensuite calculer l'indice de masse corporelle de l'utilisateur en utilisant la fonction suivante:

$$\text{IMC} = \text{poids} / \text{taille}^2$$

Le programme doit ensuite indiquer à l'utilisateur dans quelle catégorie il se trouve:

Si plus petit que 18.5, l'utilisateur est maigre

Si plus grand ou égal à 18.5 et plus petit que 25, l'utilisateur est de corpulence normale

Si plus grand ou égal à 25, l'utilisateur est obèse

Série d'exercices sur le IF

Ex3 – if_3



Écrivez un programme qui calcule la salaire hebdomadaire d'un employeur. Celui-ci est payé 23\$/heure, mais si le nombre d'heures travaillé dépasse 40 heures, chaque heures supplémentaire est payé au taux de 1.5 fois.

Le programme demande à l'employé combien d'heures il a travaillé, puis affiche son salaire.

Ex4 – if_4



Écrivez un programme qui demande à l'utilisateur d'entrer trois nombres. Le programme doit trouver quel nombre est le plus petit et l'afficher. S'il y a égalité sur le plus petit nombre, le cas doit être détecté et indiqué à l'utilisateur.

Série d'exercices sur le WHILE



Ex1 – while_1

Écrivez un programme qui crée une variable “compteur” avec une valeur égal à 0. Le programme ajoute 1 à chaque itération et s’arrête quand la variable a atteint 10. Le programme doit afficher la valeur finale de la variable.



Ex2 – while_2

Écrivez un programme qui demande à l’usager d’entrer un nombre. Le programme initialise une variable “compteur” à 0, y additionne 2 à chaque itération et affiche le résultat. Le programme ne doit s’arrêter au moment où le nombre entré par l’usager est dépassé. Par exemple:

Nombre entré : 11

Nombres affichés: 0 2 4 6 8 10

Série d'exercices sur le WHILE

Ex3 – while_3



Écrivez un programme qui initialise une variable “compteur” à 1 et y additionne 1 à chaque itération. Le programme s’arrête quand la variable atteint 10. Bien que la variable prend toutes les valeurs entre 1 et 10, le programme ne doit afficher que les valeurs impairs. Pour savoir si un nombre est pair ou impair, utilisez l’expression suivante: (nombre % 2). Cette expression retourne 0 si le nombre est pair et 1 si le nombre est impair. L’opération utilisé est celle du modulo (<http://www.cafeaulait.org/course/week2/15.html>).

Ex4 – while_4



Écrivez un programme qui demande à un usager d’entrer un nombre minimum, un nombre maximum et un interval. Le programme doit afficher tous les nombres entre le nombre minimum et maximum inclusivement en sautant de l’interval donné.

ex:

```
nombre_minimum = 10
```

```
nombre_maximum = 20
```

```
interval = 2
```

résultats à afficher: 10, 12, 14, 16, 18, 20

Série d'exercices avancé

Ex1 – avance_1

Écrivez un programme qui lance un dé aléatoire à 6 faces à l'aide de la fonction `random.nextInt`, tant que le 6 n'a pas sorti 20 fois. Il doit compter le nombre de lancers qui ont été nécessaires pour y arriver.

Ex2 – avance_2

Écrivez un programme qui lance deux dés aléatoires à 3 faces à l'aide de la fonction `random.nextInt`, de la classe `Random`. Il fait ensuite la somme des deux, puis compte combien d'occurrences de chacune des valeurs possibles: 2 à 6. Les dés sont lancés 100 fois. Le programme affiche ensuite l'identité de la valeur qui a été obtenue le plus souvent.

Ex3 – avance_3

Écrivez un programme qui demande à l'utilisateur un nombre minimum et un nombre maximum. Le programme doit compter tous les nombres pairs entre ces deux nombres. Pour savoir si un nombre est pair ou impair, utilisez l'expression suivante: `(nombre % 2)`. Cette expression retourne 0 si le nombre est pair et 1 si le nombre est impair. L'opération utilisée est celle du modulo, que nous aborderons plus tard dans le cours.

<http://www.cafeaulait.org/course/week2/15.html>

Série d'exercices sur le for



Ex1 – for_base_1

Écrivez une procédure qui contient les boucles for suivantes une à la suite de l'autre. Pour chaque boucle, le programme doit afficher chacune des valeurs du compteur.

boucle 1 départ: 1 incrément: 1 fin: 10	boucle 2 départ: 10 incrément: -1 fin: -20	boucle 3 départ: 100 incrément: 5 fin: 200	boucle 4 départ: -100 incrément: 50 fin: 100
---	--	--	--



Ex2 – for_base_2

Écrivez une procédure qui contient deux boucles for imbriquées. La première va de 1 à 100 avec des incréments de 2 et la deuxième va de la valeur du compteur de la première boucle à 100 par des incréments de 2.

Ex, si la première boucle est rendu à 25, la deuxième va de 25 à 100, en incréments de 2

Affichez les valeurs du compteurs de la second boucle.

Série d'exercices sur les classes



Ex1 – classe_base_1

Pour une classe **Pont** comprenant les attributs suivants:

- longueur (en mètres, double)
- capacite maximale (en tonnes, double)
- type de structure (String : "suspendu", "en arc", etc.)

Définissez la classe, son constructeur, ses méthodes d'accès (get/set) et une fonction d'affichage qui se comporte tel que:

Pont:

```
longueur: 1056.7 m  
capacite: 5000.0 t  
type: suspendu
```

Série d'exercices sur les classes



Ex2 – heritage_polymorphisme

Vous devez concevoir une hiérarchie de classes pour représenter des moteurs thermiques en utilisant l'héritage et le polymorphisme. Suivez les étapes ci-dessous :

Étape 1 : Classe de base MoteurThermique

Créez une classe de base nommée MoteurThermique qui contient les éléments suivants :

Attributs :

- puissance : Puissance du moteur (en kilowatts, double).
- type de carburant : Type de carburant utilisé (string).
- rendement : Rendement énergétique du moteur (en pourcentage, double).

Méthode :

- le constructeur() qui reçoit des paramètres pour assigner les valeurs aux attributs
- afficheMoteur() : affiche le moteur dans la console sous la forme :
"Moteur thermique - Puissance : X kW, Carburant : Y, Rendement : Z%".

Série d'exercices sur les classes

Étape 2 : Sous-classes spécialisées

Créez deux sous-classes spécialisées qui héritent de MoteurThermique :

MoteurEssence :

- Le type de carburant est toujours "essence".
- Le constructeur de cette classe ne demande que puissance et rendement comme arguments.
- La méthode afficheMoteur() doit afficher :

"Moteur à essence - Puissance : X kW, Rendement : Z%".

MoteurDiesel :

- Le type de carburant est toujours "diesel".
- Le constructeur de cette classe ne demande que puissance et rendement comme arguments.
- La méthode afficheMoteur() doit afficher :

"Moteur Diesel - Puissance : X kW, Rendement : Z% - Plus adapté aux charges lourdes.".